SESSION 2 OF 8

Writing Better Code Faster

with Copilot

DAY 1 — Foundations of AI-Augmented Engineering

Duration: 90 Minutes | Hands-on

Anurag Kumar

QUICK RECAP

Session 1 Takeaways + Where We Are

What we learned in Session 1:

- Copilot is a collaborator, not autocomplete
- Context quality determines output quality
- Three modes: Ask (learn), Agent (build), Plan (blueprint)
- Use #codebase and #file for targeted context

NOW

Session 2

From asking questions
to building code

Session 1: You learned to talk to the codebase. Session 2: You learn to build with it.

SESSION GOAL

Guide Copilot to produce production-quality code

using prompt-driven development

The shift: from asking Copilot questions to giving it specifications.

SESSION AGENDA

Use Copilot to write actual code not just edit or refactor.

01 **Prompt Patterns for Coding**
Instruction, constraint, example-driven, role-based

02 **Comment-as-Spec Development**
Write comments first, Copilot generates implementation

03 **Custom Instructions**
copilot-instructions.md, AGENTS.md, org-level

04 **Ask vs Agent for Code Gen**
When to use each mode for building code

05 **Generate → Review → Refine → Test**
The iterative development cycle with AI

PROMPT PATTERNS

Four Ways to Talk to Copilot When Coding

Instruction

Tell Copilot exactly what to build

"Create a Spring Boot service for processing fund transfers with validation and logging."

Constraint

Specify boundaries and limitations

"Generate this service but do NOT use Lombok. Use BigDecimal for all monetary values."

Example-Driven

Show input/output patterns

"Given input {fromId: 1, toId: 2, amount: 500}, return {referenceId: UUID, status: COMPLETED}"

Role-Based

Set a persona for quality

"As a senior Java developer at a bank, review and improve this transfer validation logic."

COMMENT-AS-SPEC

Write the Spec First, Let Copilot Implement

The Pattern:

Write detailed comments describing what the code should do. Copilot reads them as a specification and generates the implementation.

```
// Service: AccountService
// Method: searchByHolderName
// - Accept partial name string
// - Case-insensitive search
// - Return List<AccountDTO>
// - Only return ACTIVE accounts
// - Log search query at DEBUG level

// Copilot generates from here ↓
```

Key Insight

Comment quality = Code quality

Vague comments get vague code. Specific comments with constraints get production-quality implementations.

CUSTOM INSTRUCTIONS

Teaching Copilot Your Project Standards

Before you write a single prompt, give Copilot your team's playbook.

Without custom instructions:

Copilot gives generic Java suggestions. You correct it every time.

With custom instructions:

Copilot knows your stack, conventions, and domain. Every response is on target.

What to put in your instructions:

Tech stack	Java 17, Spring Boot 3.2+, Maven, H2
Coding conventions	BigDecimal for money, DTOs not entities, SLF4J logging
Naming standards	{Domain}Service, {Domain}Controller, {Action}Request
Security rules	Never hardcode secrets, validate all input, use Bean Validation
Domain context	Banking API, account format FIS-XXXX-XXXX, ACTIVE/FROZEN/CLOSED

THREE LEVELS

Workspace, Repository, and Organization

WORKSPACE

`.github/copilot-instructions.md`

Auto-applies to all chat requests in this workspace. One file, stored in the repo. Your team's coding playbook.

Best for:

Project-specific conventions, tech stack, patterns

MULTI-AGENT

`AGENTS.md`

Works with Copilot, Claude Code, Codex, and others. Place at root or in subfolders for scoped rules.

Best for:

Multi-agent workflows, folder-specific rules

ENTERPRISE

`Organization Instructions`

Defined at GitHub org level. Shared across all repos and workspaces in the organization.

Best for:

Enterprise-wide standards at FIS (security, compliance)

CHOOSING YOUR MODE

Ask vs Agent for Code Generation

Ask Mode

YOU CONTROL

- Generates code in the chat panel
- You copy/paste what you want
- No files modified automatically
- Full control, more manual work

Use when: exploring ideas, comparing approaches, learning patterns

Agent Mode

AI DRIVES

- Edits files directly in your project
- Creates new files if needed
- Runs terminal commands
- Self-corrects on errors, loops until done

Use when: building features, scaffolding, multi-file changes

THE WORKFLOW

Generate → Review → Refine → Test

Never accept the first output. Iterate.

1

Generate

Give Copilot a clear prompt with context and constraints

→

2

Review

Read the output critically. Does it match your intent?

→

3

Refine

Follow up: “Add error handling”, “Use BigDecimal”, “Rename this”

→

4

Test

Compile, run, verify. Ask Copilot to generate tests too.

↻ **Repeat until production-quality**

USE CASES

Real-World Scenarios at FIS

DEVELOPER

Service from Spec

Write a structured comment describing TransactionService. Copilot generates the full service class layer by layer.

DEVELOPER

REST Endpoints

Generate Spring Boot controller with validation, logging, and error handling using a single structured prompt.

TEAM

Comment-as-Spec Adoption

User stories written as code comments. Copilot generates corresponding implementations. Entire team workflow shifts.

DEVELOPER

Iterative Enhancement

Start with a basic fraud check stub. Progressively add rules, thresholds, and keyword detection through follow-up prompts.

HANDS-ON TIME

Building with Copilot

Tools: VS Code + Banking API + Copilot Chat (Ask & Agent modes)

EXERCISE 0

Create copilot-instructions.md

What to do:

As a group, create .github/copilot-instructions.md in your project. This file tells Copilot your tech stack, coding conventions, naming standards, and domain context. Every exercise after this benefits from it.

Try this prompt:

```
Create a file at .github/copilot-instructions.md
with the following sections:
- Tech Stack (Java 17, Spring Boot, Maven, H2)
- Coding Conventions (BigDecimal, DTOs, SLF4J)
- Naming Standards ({Domain}Service, etc.)
- Banking Domain (account format, status rules)
```

Pro Tip: After creating this file, try asking Copilot the same question from Session 1. Notice how the response quality improves — it now knows your conventions.

EXERCISE 1

Comment-as-Spec: Generate AccountService

What to do:

Open AccountService.java (currently only has getAccountById). Write structured comments describing listAllAccounts and searchByHolderName, then let Copilot generate the implementations.

Try this prompt:

```
// Method: listAllAccounts
// - Return List<AccountDTO> of all ACTIVE accounts
// - Sort by holderName ascending
// - Log count at INFO level

// Method: searchByHolderName(String name)
// - Partial match, case-insensitive
// - Only ACTIVE accounts
// - Return empty list if no matches
```

Pro Tip: Write the comments first, then position your cursor below and wait for Copilot inline. Compare this with asking in Chat.

EXERCISE 2

Agent Mode: Generate AccountController

What to do:

Switch to Agent mode. Ask Copilot to generate a complete REST controller for the Account endpoints with proper error handling and DTOs.

Try this prompt:

```
Generate a Spring Boot REST controller called
AccountController with endpoints:
- GET /api/accounts (list all active accounts)
- GET /api/accounts/{id} (lookup by ID, 404 if missing)
- GET /api/accounts/search?holder={name} (search)
Include proper ResponseEntity handling and
use AccountDTO for all responses.
```

Pro Tip: Watch Agent mode create the file, add imports, and wire up the service. Review every line before accepting.

EXERCISE 3

Add Validation to TransferRequest DTO

What to do:

Open TransferRequest.java. The fields exist but have no validation. Ask Copilot to add Jakarta Bean Validation annotations with specific business rules.

Try this prompt:

```
Add Jakarta Bean Validation to TransferRequest:
- fromAccountId: @NotNull
- toAccountId: @NotNull
- amount: @NotNull, @DecimalMin("1.00"),
  @DecimalMax("50000.00")
- currency: @NotBlank, @Size(min=3, max=3)
- description: @Size(max=255), optional
```

Pro Tip: Notice how Copilot respects the constraints from copilot-instructions.md — it should already know to use BigDecimal.

EXERCISE 4

Iteratively Enhance FraudCheckService

What to do:

Start with the basic fraud stub. Layer on rules one at a time using follow-up prompts. This demonstrates iterative refinement — the core skill of prompt-driven development.

Try this prompt:

```
Prompt 1: "Enhance FraudCheckService to flag
transfers above $10,000 with risk score 50."

Prompt 2: "Now add keyword detection for
'urgent', 'wire immediately'. Risk score 75."

Prompt 3: "Add a rule: flag if destination account
received 5+ transfers in the last hour."
```

Pro Tip: Each follow-up builds on the previous output. This is how real AI-assisted development works — progressively, not in one shot.

EXERCISE 5

Progressive Refinement: Logging, Safety, Pagination

What to do:

Take the AccountService you generated in Exercise 1. Progressively improve it with three follow-up prompts. Each one adds a production concern.

Try this prompt:

```
Prompt 1: "Add SLF4J logging to every method.
Log entry at DEBUG, results at INFO."

Prompt 2: "Add null-safety and input validation.
Return empty list for null/blank search terms."

Prompt 3: "Add pagination support to
listAllAccounts using Spring's Pageable."
```

Pro Tip: This is the Generate → Review → Refine → Test cycle in action. After each prompt, review before prompting again.

PROMPT REFERENCE

Session 2 — All Key Prompts

0
```
Create .github/copilot-instructions.md with tech stack, coding conventions, naming standards, and domain context.
```

1
```
// Method: listAllAccounts — Return List<AccountDTO> of ACTIVE accounts, sorted by holderName, log count at INFO.
```

2
```
Generate AccountController with GET endpoints for list, lookup, and search. Use ResponseEntity and AccountDTO.
```

3
```
Add Jakarta Bean Validation to TransferRequest: @NotNull, @DecimalMin, @DecimalMax, @Size constraints.
```

4
```
Enhance FraudCheckService: flag transfers above $10K (score 50), suspicious keywords (score 75), high-frequency (5+/hour).
```

5
```
Add SLF4J logging, null-safety, and pagination support to AccountService.
```

Prompt-Driven Development

The quality of your prompts directly determines the quality of Copilot's output.
Treat prompts as engineering specifications.

Reflection

- Which prompt pattern felt most natural to you?
- Where did you need to refine Copilot's first output?
- How would copilot-instructions.md change your team's workflow?

UP NEXT

Session 3

AI for Requirements
& Functional Analysis